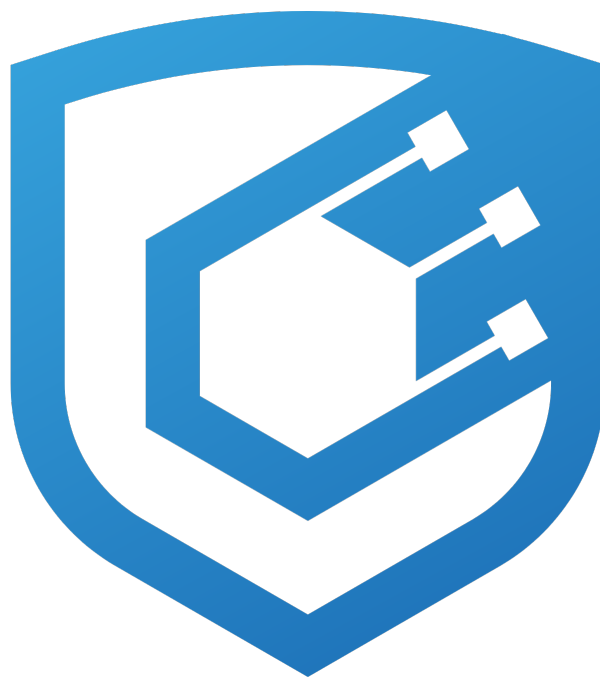




Myriad PR151 Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Immeas](#)

[Kiki](#)

June 5, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
6.1	Centralization Risks	3
6.2	Defensive Improvements	3
7	Findings	6
7.1	Medium Risk	6
7.1.1	NegRiskAdapter::redeemNOPositions permanently bricks on a voided neg-risk event when redeemVoided floor-rounding leaves recovered wcol below minted	6
7.1.2	Partial admin resolve bricks void and CLOB	8
7.2	Low Risk	10
7.2.1	NegRiskAdapter::voidEvent permits a full YES payout with zero NO payout that permanently reverts redeemNOPositions via the ConditionalTokens::redeemVoided zero-payout guard	10
7.2.2	NegRiskAdapter::resolveEventMarket permissionless per-market resolution permanently forecloses voidEvent for the entire event	11
7.2.3	NegRiskAdapter::resolveEventMarket first-mover race permanently wedges a second YES-returning oracle market, blocking resolveEvent and redemption	12
7.2.4	PR-151 deploy and interaction scripts load the broadcasting private key from a plaintext PRIVATE_KEY env var	13
7.2.5	Per-market admin mutators adminSetClosesAt and updateMarketOracle lack the neg-risk carve-out enforced on the resolution and void paths	14
7.3	Informational	16
7.3.1	redeemNOPositions lost its NatSpec in the PR and the new resolution entypoints document non-obvious caller and invariant semantics inconsistently	16
7.3.2	Bad debt socialize comment is misleading	16
7.3.3	NegRiskAdapter::convertPositions routes a converter's deposited NO stake to the treasury on an all-NO event resolution	16
7.3.4	winningIndex -2 wrong after void	17
7.4	Gas Optimization	18
7.4.1	NegRiskAdapter::voidEvent redundantly re-writes winningIndex with the unchanged -2 sentinel	18

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Myriad is an on-chain prediction-market exchange. `MyriadCTFExchange` is a central-limit order book that settles EIP-712-signed maker/taker orders into ERC1155 outcome tokens minted by a `ConditionalTokens` contract, with `PredictionMarketV3ManagerCLOB` managing market lifecycle and resolution. `NegRiskAdapter` coordinates “neg-risk” events, where several mutually-exclusive binary markets share one event under an “at most one YES per event” invariant, and `WrappedCollateral` (`wcol`) is a 1:1 ERC20 wrapper around the underlying collateral (USDC.e) that the conditional-tokens layer uses as its collateral asset.

This review covered the non-CRE changes in PR-151, which hardened the neg-risk subsystem: the exchange now lets traders pay in underlying while bridging to `wcol` for conditional-token split/merge (via `NegRiskAdapter::wrapForExchange` and `WrappedCollateral::unwrap`); `WrappedCollateral::wrap` became adapter-only; per-market and event-level resolution were made permissionless through `NegRiskAdapter::resolveEvent`, `resolveEventMarket` (routed through `PredictionMarketV3ManagerCLOB::resolveMarket`); and `voidEvent` and `redeemNOPositions` gained payout-cap and solvency-assertion guards. The CRE oracle contracts added by the same PR were out of scope.

5 Audit Scope

The audit scope was limited to non-CRE changes in

```
// see https://github.com/Polkamarkets/polkamarkets-js/pull/151/changes for full changes

// minor targeted changes to existing
contracts/MyriadCTFExchange.sol
contracts/PredictionMarketV3ManagerCLOB.sol
contracts/WrappedCollateral.sol

// more significant changes to existing
contracts/NegRiskAdapter.sol
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the [PR151](#) code provided by [Myriad](#). The findings consist of 2 Medium & 5 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

The protocol relies on several privileged roles, administered through `AdminRegistry`, that holders must be trusted with. `DEFAULT_ADMIN_ROLE` can redeem the adapter's NO positions and route residual collateral to the treasury. `RESOLUTION_ADMIN_ROLE` can resolve, override, and void neg-risk events and individual markets. `MARKET_ADMIN_ROLE` can create markets. The exchange's `negRiskAdapter` pointer, the adapter's exchange pointer, and `WrappedCollateral`'s adapter are admin-set wiring with no on-chain consistency enforcement between them. Holders of these roles should be treated as fully trusted; a compromised or misconfigured admin can disrupt resolution and redemption flows.

Specific admin-trust surfaces introduced or widened by PR-151:

- `NegRiskAdapter::voidEvent` caps the YES-payout basket at a total of one unit but enforces no floor tied to the realized outcome. A resolution admin can void an event with an under-allocated YES basket, so the unallocated residual is swept to the treasury rather than to the YES holders of the voided markets. The contract cannot distinguish a legitimate "Other won, refund is protocol surplus" void from one that under-pays rightful winners; the correctness of every void rests on admin policy.
- Neg-risk per-market and event resolution is now permissionless and sources its outcome entirely from each market's pluggable oracle, with no second source and no dispute or finalization window. The integrity of resolution rests on the trusted configuration of those oracles.
- A signed order binds the market but not the settlement token; the token is resolved at fill time from the admin-mutable `negRiskAdapter` pointer. An admin reconfiguration between signing and filling can flip a pending order's settlement token, and an unset or mismatched adapter pointer can route settlement to a token traders cannot supply. These are admin/wiring-policy surfaces rather than externally triggerable bugs, but they have no code-level guard today.

6.2 Defensive Improvements

The following defense-in-depth hardening would reduce the blast radius of misconfiguration and admin error around the neg-risk changes:

- Enforce the neg-risk wiring at the code level: have `MyriadCTFExchange` revert (rather than silently fall back to wcol) when the manager reports a market as neg-risk but the exchange's `negRiskAdapter` is unset, and assert consistency across the exchange's `negRiskAdapter`, the manager's `negRiskAdapter`, `WrappedCollateral`'s adapter, and the adapter's underlying at configuration time and on each settlement path.
- Bind the settlement token (or the neg-risk flag) into the signed order hash so a pending order cannot be re-pointed to a different token by an admin reconfiguration between sign-time and fill-time.
- Constrain `voidEvent` so that void payouts are provably make-whole for the realized outcome (for example, require the basket to sum to exactly one unit, or gate the "route residual to treasury" path behind an explicit "no named outcome won" flag) so that voiding cannot silently divert winners' principal.
- Add a finalization or dispute window to permissionless neg-risk resolution so a single oracle report cannot be locked in irreversibly without an opportunity for admin override.

Summary

Project Name	PR151
Repository	polkamarkets-js
Commit	617c40dbf24b...
Fix Commit	a6306bd6d8bb...
Audit Timeline	Jun 1st - Jun 2nd, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	5
Informational	4
Gas Optimizations	1
Total Issues	12

Summary of Findings

[M-1] <code>NegRiskAdapter::redeemNOPositions</code> permanently bricks on a voided neg-risk event when <code>redeemVoided</code> floor-rounding leaves recovered wcol below minted	Resolved
[M-2] Partial admin resolve bricks void and CLOB	Resolved
[L-1] <code>NegRiskAdapter::voidEvent</code> permits a full YES payout with zero NO payout that permanently reverts <code>redeemNOPositions</code> via the <code>ConditionalTokens::redeemVoided</code> zero-payout guard	Resolved
[L-2] <code>NegRiskAdapter::resolveEventMarket</code> permissionless per-market resolution permanently forecloses <code>voidEvent</code> for the entire event	Resolved
[L-3] <code>NegRiskAdapter::resolveEventMarket</code> first-mover race permanently wedges a second YES-returning oracle market, blocking <code>resolveEvent</code> and redemption	Resolved
[L-4] PR-151 deploy and interaction scripts load the broadcasting private key from a plaintext <code>PRIVATE_KEY</code> env var	Resolved
[L-5] Per-market admin mutators <code>adminSetClosesAt</code> and <code>updateMarketOracle</code> lack the neg-risk carve-out enforced on the resolution and void paths	Resolved
[I-1] <code>redeemNOPositions</code> lost its <code>NatSpec</code> in the PR and the new resolution endpoints document non-obvious caller and invariant semantics inconsistently	Resolved
[I-2] Bad debt socialize comment is misleading	Resolved

[I-3] <code>NegRiskAdapter::convertPositions</code> routes a converter's deposited NO stake to the treasury on an all-NO event resolution	Resolved
[I-4] <code>winningIndex -2</code> wrong after void	Resolved
[G-1] <code>NegRiskAdapter::voidEvent</code> redundantly re-writes <code>winningIndex</code> with the unchanged <code>-2</code> sentinel	Resolved

7 Findings

7.1 Medium Risk

7.1.1 `NegRiskAdapter::redeemNOPositions` permanently bricks on a voided neg-risk event when `redeemVoided` floor-rounding leaves recovered wcol below `minted`

Description: `NegRiskAdapter::redeemNOPositions` was changed by PR-151 to replace a tolerant `min(minted, wcolRecovered)` burn with a strict solvency assertion: `require(wcolRecovered >= minted, "insolvent event payouts")` (contracts/NegRiskAdapter.sol:505). The `minted` value is `mintedWcolPerEvent[eventId]` - an exact integer equal to $(n-1) * \text{amount accumulated by } \text{convertPositions} \text{ and } \text{mintAllYesTokens}$. The `wcolRecovered` value is built market-by-market by calling `ConditionalTokens::redeemVoided` (out of scope, traced for context), which applies floor division: `totalPayout += (outcomeBalance * outcomePayout) / 1e18`.

For a voided event the adapter holds NO tokens in every constituent market; each NO payout is $1e18 - \text{yesPayouts}[i]$ as set by `voidEvent` (contracts/NegRiskAdapter.sol:468). The event-level solvency cap in `voidEvent` - `require(totalYesPayout <= 1e18, "event payouts overallocated")` at contracts/NegRiskAdapter.sol:462 - explicitly endorses the full-basket case where the sum equals exactly $1e18$ as the maximum permitted void. At that boundary the exact (real-number) recovery equals `minted`, but the per-market floor in `redeemVoided` strips up to 1 wei from each of the n markets. The resulting `wcolRecovered` lands below `minted` whenever any term $T * (1e18 - \text{yesPayouts}[i])$ is not an exact multiple of $1e18$ - the ordinary condition for fractional void ratios. The `require` then reverts.

This sub-wei floor dust is the most common trigger, but the strict assertion reverts on a shortfall of any size: a market whose floored recovery is zero (a near- $1e18$ YES payout against a realistic or deliberately-seeded small NO balance) contributes nothing to `wcolRecovered` while its full unbacked share still counts toward `minted`, so the shortfall - and the permanent revert - can be arbitrarily large rather than dust. Because `redeemNOPositions` is one-shot (gated on `require(!noPositionsRedeemed[eventId])` at contracts/NegRiskAdapter.sol:478) and the void payouts are immutable after `voidEvent` sets `evt.resolved = true` (contracts/NegRiskAdapter.sol:464), the revert is permanent with no retry path.

Files:

`NegRiskAdapter::redeemNOPositions`, `voidEvent` - contracts/NegRiskAdapter.sol:454-521

Impact: `redeemNOPositions` is permanently DoS'd for any voided neg-risk event whose YES payouts sum near $1e18$ and include fractions that do not divide the per-market NO-token balance evenly - the common case for real-world fractional void allocations. The consequences are:

- The adapter's recovered NO collateral remains locked in the adapter.
- The `minted` unbacked wcol created by `adapterMint` is never burned by `adapterBurn`, so `WrappedCollateral` circulating supply permanently exceeds its underlying backing by `minted` - the exact under-collateralization the new assertion was intended to prevent.
- The treasury never receives its excess.
- `noPositionsRedeemed[eventId]` stays false forever.

Because `WrappedCollateral::unwrap` draws from a shared underlying pool, the shortfall is borne by the last holders to unwrap. No user funds are directly seized; the harm is a persistent unbacked-wcol liability and a locked treasury sweep, recoverable only if the protocol can deploy a patched contract. The magnitude of the locked amount scales with the per-event `minted` balance.

Reachability note: the brick is partly grieveable. On a barely-utilized event, an attacker can seed a small NO balance in a market that a later, otherwise-reasonable void floors to zero recovery; that market's unbacked `minted` contribution then cannot be recovered, so the redemption reverts regardless of any fixed per-market rounding tolerance. The trigger does not require the attacker to control the void payouts.

Proof of Concept: Consider a 3-outcome neg-risk event where `mintAllYesTokens` was called with `amount = 1_000_000` (1 USDC at 6 decimals). The adapter holds NO balance of 1,000,000 in each of the three markets; `minted = (3-1) * 1_000_000 = 2_000_000`.

An admin calls voidEvent with yesPayouts = [3333333333333333, 3333333333333333, 3333333333333334] - a natural equal-thirds split summing to exactly 1e18, which passes both per-element and aggregate caps (contracts/NegRiskAdapter.sol:456, 462).

voidEvent sets NO payouts to [6666666666666667, 6666666666666667, 6666666666666666] via admin-VoidMarket (contracts/NegRiskAdapter.sol:468).

redeemNOPositions calls ConditionalTokens::redeemVoided for each market:

- market 0: $\text{floor}(1_000_000 * 6666666666666667 / 1e18) = 666,666$
- market 1: $\text{floor}(1_000_000 * 6666666666666667 / 1e18) = 666,666$
- market 2: $\text{floor}(1_000_000 * 6666666666666666 / 1e18) = 666,666$

wcolRecovered = 1_999_998. require(1_999_998 >= 2_000_000) reverts with "insolvent event payouts". Every subsequent call reverts identically.

Recommended Mitigation: Check solvency at void time on un-floored products, and make redemption skip-tolerant with a bounded dust allowance, so a genuinely insolvent void is rejected at voidEvent (recoverable) while rounding never bricks the one-shot redeemNOPositions.

1. Reject insolvent voids in voidEvent. Sum the un-floored recovery products and compare against minted scaled by 1e18 - summing before the single division removes the per-market rounding loss that an after-the-fact tolerance would otherwise have to absorb:

```
uint256 totalProduct;
for (uint256 i = 0; i < n; i++) {
    uint256 mid = evt.marketIds[i];
    totalProduct += conditionalTokens.balanceOf(address(this), conditionalTokens.getTokenId(mid,
    ↪ Outcomes.NO)) * (1e18 - yesPayouts[i]);
    totalProduct += conditionalTokens.balanceOf(address(this), conditionalTokens.getTokenId(mid,
    ↪ Outcomes.YES)) * yesPayouts[i];
}
require(totalProduct >= mintedWcolPerEvent[eventId] * 1e18, "void leaves event insolvent");
```

2. In redeemNOPositions, skip a market only when both its YES and NO floored recoveries are zero (matching redeemVoided's own logic, so it cannot revert with "zero payout"), then reconcile with a dust-bounded check and burn what was recovered:

```
// per market:
(uint256 yesPay, uint256 noPay) = manager.getVoidedPayouts(mid);
uint256 yesBal = conditionalTokens.balanceOf(address(this), conditionalTokens.getTokenId(mid,
↪ Outcomes.YES));
uint256 noBal = conditionalTokens.balanceOf(address(this), conditionalTokens.getTokenId(mid,
↪ Outcomes.NO));
if ((yesBal * yesPay) / 1e18 + (noBal * noPay) / 1e18 == 0) continue;
conditionalTokens.redeemVoided(mid);

// after the loop:
uint256 toBurn = wcolRecovered < minted ? wcolRecovered : minted;
require(wcolRecovered + 2 * n >= minted, "insolvent beyond rounding"); // 2n wei = max floor dust
if (toBurn > 0) wcol.adapterBurn(address(this), toBurn);
uint256 excess = wcolRecovered - toBurn; // surplus -> unwrap to treasury
```

Notes:

- The redeem-time allowance is bounded floor dust (each market floors at most its YES and NO term, under 1 wei each), so it must be an O(n)-wei constant such as 2 * n - not n * 1e18, which would tolerate roughly n whole tokens of genuine insolvency.
- The void-time projection and the redeem-time skip must use the same both-sided recovery formula as redeemVoided.

- Any residual `minted - wcolRecovered` (bounded dust given the void-time check) is unburned unbacked `wcol`; track it as protocol-owed and cover it explicitly rather than leaving it to block the last `unwrap`.
- `voidEvent` becomes an $O(n)$ loop with per-market balance reads, which is acceptable for an admin-only call.

Myriad: Fixed in commits [9e78913](#) and [41ca647](#)

Cyfrin: Verified. The strict `require(wcolRecovered >= minted)` that caused the brick is removed, and genuine insolvency is now rejected up front at `voidEvent` time against an un-floored product sum (`sum(noBal*(1e18-yesPayouts) + yesBal*yesPayouts) >= minted*1e18`), so per-market floor-division dust can no longer revert the one-shot `redeemNOPositions`.

7.1.2 Partial admin resolve bricks void and CLOB

Description: Neg-risk events group several binary markets under one `eventId`. Resolution admin can resolve a single leg early through `adminResolveEventMarket`, which calls `PredictionMarketV3ManagerCLOB::adminResolveMarket`. Unlike permissionless `resolveMarket` and unlike `PredictionMarketV3ManagerCLOB::adminVoidMarket`, that admin path does not require `block.timestamp >= closesAt`. The adapter does not set `evt.resolved` when only one leg is terminal, so the event still looks open at the adapter layer while one market is already resolved.

Once any leg is resolved, `NegRiskAdapter::voidEvent` cannot run. It loops all constituent markets through `adminVoidMarket`, which reverts with `resolved` on the leg that was already settled. Event-level void with custom YES and NO payout ratios becomes unavailable for the remainder of the event lifetime.

Cross-market exchange flow breaks for the same reason. `MyriadCTFExchange::matchCrossMarketOrders` requires a BUY YES order on every outcome and calls `_requireMarketOpen` on each leg. A resolved leg returns the revert reason `MarketNotTradeable`.

`convertPositions` only splits legs other than the NO leg the user supplies. If the resolved leg sits in the split set, the transaction reverts. Convert from the resolved leg's NO can still succeed because that leg is excluded from the split loop.

Impact:

- Event cannot be voided - one resolved leg blocks `voidEvent` for the entire event; custom void refund ratios cannot be applied.
- Trading continues on a "cancelled" event - per-market CLOB on open legs still works.
- Capital efficiency is throttled down due to the limited settlement options. Making the event less desirable as orders are not matched effectively

Proof of Concept: Consider a four-outcome neg-risk event - outcomes A, B, C, and D - created with `createEvent`. Markets are still before `closesAt`. Resolution admin decides outcome B is eliminated and calls `adminResolveEventMarket` with `outcomeIndex` for B and outcome NO. `PredictionMarketV3ManagerCLOB` marks only B's market resolved. The adapter never sets `evt.resolved`, so the parent event still reads as open while B is terminal. For the rest of the event lifetime, the main settlement and trading paths below fail until every leg is resolved through other admin paths or contracts change.

1. **Early admin resolve on B.** Call `adminResolveEventMarket` on outcome B while A, C, and D remain open. B is resolved; siblings are not. `getEvent` still shows `resolved == false` on the adapter.
2. **Cross-market bundle mint.** With B settled, submit a valid `matchCrossMarketOrders` bundle with one BUY YES per outcome across A-D. The exchange requires all legs tradeable; B returns `MarketNotTradeable`. Neg-risk parity minting for the full set fails for the duration of the event.
3. **User `convertPositions` from an open leg.** A holder calls `convertPositions` supplying NO on outcome C. The adapter must split A, B, and D. The split on B reverts for the same reason. Convert from C fails even though C itself is still open. Convert supplying only B's NO can still run because B is excluded from the split set - a narrow escape hatch, not a full event settlement path.

4. **Post-close** voidEvent. After closesAt passes on all legs, resolution admin attempts voidEvent with custom YES payout ratios across A-D. The loop calls adminVoidMarket on each market; the call on B reverts resolved. Event-level void never succeeds. Custom void refunds remain unavailable.

Recommended Mitigation: A complete fix that supports progressive admin resolution while preserving voidEvent, convertPositions, and matchCrossMarketOrders would require architectural changes.

As a narrower mitigation, add `require(block.timestamp >= market.closesAt)` to `adminResolveMarket` so admin resolution matches permissionless resolve and void timing. That prevents settlement paths from breaking and keeps voidEvent available.

Myriad: Fixed in commits [f8b1080](#), [46f2696](#), and [82818a7](#)

Cyfrin: Verified.

7.2 Low Risk

7.2.1 `NegRiskAdapter::voidEvent` permits a full YES payout with zero NO payout that permanently reverts `redeemNOPositions` via the `ConditionalTokens::redeemVoided` zero-payout guard

Description: `NegRiskAdapter::voidEvent` validates each per-market payout with `require(yesPayouts[i] <= 1e18, "yes payout > 1e18")` (`contracts/NegRiskAdapter.sol:456`) and the event-level aggregate with `require(totalYesPayout <= 1e18, "event payouts overallocated")` (`contracts/NegRiskAdapter.sol:462`). A configuration such as `[1e18, 0, 0]` (`sum = 1e18`) passes both checks. `voidEvent` then calls `manager.adminVoidMarket(marketId, yesPayouts[i], 1e18 - yesPayouts[i])` for each constituent market (`contracts/NegRiskAdapter.sol:468`). For the market where `yesPayouts[i] == 1e18`, this sets its NO-outcome payout to `1e18 - 1e18 = 0`.

After any `mintAllYesTokens` or `convertPositions` call the adapter holds NO tokens in every constituent market (the YES tokens are distributed to recipients). When `redeemNOPositions` iterates the markets and reaches the market voided with NO payout 0, it calls `ConditionalTokens::redeemVoided` (`contracts/NegRiskAdapter.sol:493`). Inside `redeemVoided`, the adapter's NO balance is non-zero but `outcome1Payout == 0`, so `totalPayout = (balance * 0) / 1e18 = 0` (`contracts/ConditionalTokens.sol`), and the function reverts on `require(totalPayout > 0, "zero payout")` at `contracts/ConditionalTokens.sol`. This reverts the entire `redeemNOPositions` transaction. Because `redeemNOPositions` processes all markets in one atomic transaction and is one-shot (`require(!noPositionsRedeemed[eventId])` at `contracts/NegRiskAdapter.sol:478`), and the void payouts are immutable once set, there is no way to skip the offending market or retry. The minted unbacked wcol is never burned and the treasury sweep never executes.

Files:

`NegRiskAdapter::voidEvent`, `redeemNOPositions` - `contracts/NegRiskAdapter.sol:441-521`; `ConditionalTokens::redeemVoided` - `contracts/ConditionalTokens.sol`

Impact: A `RESOLUTION_ADMIN_ROLE` who voids a multi-outcome event with a configuration that assigns all payout to one named outcome - the natural "outcome X won, fully refund X holders" case - permanently disables `redeemNOPositions` for that event. The unbacked minted wcol remains in `WrappedCollateral` supply indefinitely, causing circulating wcol to exceed the underlying pool. The treasury never receives its excess. Because this is triggered by a configuration that is semantically natural and passes all current validation guards, it can arise under honest operation. The impact is permanent DoS of the adapter's cleanup for the affected event and a persistent wcol under-collateralization proportional to `mintedWcolPerEvent[eventId]`. No user funds are directly taken; the harm is the unretired unbacked liability and locked cleanup path.

Recommended Mitigation: The condition that bricks redemption is a market whose FLOORED recovery is zero, `noBalance * noPayout < 1e18` (with `noPayout = 1e18 - yesPayouts[i]`). This includes, but is broader than, a NO payout of exactly zero: a small nonzero NO payout against a realistic balance also floors to zero (for example `[1e18 - 1, 1, 0]`, where market 0's NO payout of 1 makes `floor(noBalance * 1 / 1e18) = 0`). Because the condition depends on the runtime balances, address it where the balances are known, in two coordinated parts.

First, in `redeemNOPositions`, skip a market only when both its YES and NO floored recoveries are zero - matching `redeemVoided`'s own logic, so the zero-payout guard cannot revert the batch (the both-sided test also covers the rare case of the adapter holding YES tokens):

```
(uint256 yesPay, uint256 noPay) = manager.getVoidedPayouts(mid);
uint256 yesBal = conditionalTokens.balanceOf(address(this), conditionalTokens.getTokenId(mid,
↳ Outcomes.YES));
uint256 noBal = conditionalTokens.balanceOf(address(this), conditionalTokens.getTokenId(mid,
↳ Outcomes.NO));
if ((yesBal * yesPay) / 1e18 + (noBal * noPay) / 1e18 == 0) continue;
conditionalTokens.redeemVoided(mid);
```

Second, because a skipped market contributes nothing to recovery while its unbacked wcol is still counted in minted, enforce solvency at void time using an un-floored product sum (summing before the single division so rounding never rejects a solvent void):

```
uint256 totalProduct;
for (uint256 i = 0; i < n; i++) {
```

```

uint256 mid = evt.marketIds[i];
totalProduct += conditionalTokens.balanceOf(address(this), conditionalTokens.getTokenId(mid,
↳ Outcomes.NO)) * (1e18 - yesPayouts[i]);
totalProduct += conditionalTokens.balanceOf(address(this), conditionalTokens.getTokenId(mid,
↳ Outcomes.YES)) * yesPayouts[i];
}
require(totalProduct >= mintedWcolPerEvent[eventId] * 1e18, "void leaves event insolvent");

```

An insolvent or zero-flooring configuration then reverts at `voidEvent` call time (recoverable - the admin picks a different distribution) rather than permanently bricking the one-shot `redeemNOPositions`. A `voidEvent`-input guard on `yesPayouts[i]` alone cannot express this, since the brick depends on the runtime balances.

Myriad: Fixed in commits [9e78913](#) and [41ca647](#)

Cyfrin: Verified. `redeemNOPositions` now skips a voided market when both its YES and NO floored recoveries are zero, matching `redeemVoided`'s own logic, so the `require(totalPayout > 0, "zero payout")` guard can no longer revert the one-shot redemption; the companion un-floored void-time solvency check accounts for the skipped markets' unbacked share.

7.2.2 `NegRiskAdapter::resolveEventMarket` **permissionless per-market resolution permanently forecloses voidEvent for the entire event**

Description: `NegRiskAdapter::resolveEventMarket` is fully permissionless - any caller can invoke it for any constituent market of a neg-risk event once the market's `closesAt` has passed (contracts/`NegRiskAdapter.sol`:329-340). It calls `manager.resolveMarket(marketId)`, which permanently transitions the market to `MarketState::resolved` with a YES or NO outcome derived from the per-market pluggable oracle (out-of-scope `IMarketOracle`). Once even a single constituent market is resolved through this path, `voidEvent` becomes permanently inaccessible for the entire event.

`voidEvent` calls `manager.adminVoidMarket` for every constituent market (contracts/`NegRiskAdapter.sol`:467-469). `adminVoidMarket` requires `market.state != MarketState.resolved` at `PredictionMarketV3ManagerCLOB.sol`:224. A market resolved by `resolveEventMarket` is already in `MarketState::resolved`, so `adminVoidMarket` reverts on it, and `voidEvent` reverts for the whole event. There is no skip branch in `voidEvent` for already-resolved markets.

The two admin resolution remedies are similarly foreclosed. `adminResolveEventMarket` calls `manager.adminResolveMarket`, which also enforces `require(market.state != MarketState.resolved, "resolved")` at `PredictionMarketV3ManagerCLOB.sol`:200 - so it cannot re-resolve an already-resolved constituent market. `adminResolveEvent` does tolerate already-resolved markets (it checks consistency rather than reverting unconditionally, per contracts/`NegRiskAdapter.sol`:427-430), but `adminResolveEvent` requires `!evt.resolved` (contracts/`NegRiskAdapter.sol`:413) and sets `evt.resolved = true` atomically. An event where one constituent is permissionlessly resolved to YES and others remain unresolved can still be finalized by `adminResolveEvent` - but if the event was intended for voiding (e.g. disputed outcome, late-breaking ambiguity), the one-way `adminResolveEvent` path commits to a winner rather than voiding, providing no path to the refund semantics that `voidEvent` would have delivered.

The result is that a single permissionless `resolveEventMarket` call - executable by any externally owned account with no access control - permanently removes the `RESOLUTION_ADMIN_ROLE`'s ability to void the event, even when `evt.resolved` is still false at the time the call is made.

Files:

```

NegRiskAdapter::voidEvent, resolveEventMarket - contracts/NegRiskAdapter.sol:329-340, 441-472;
PredictionMarketV3ManagerCLOB::adminVoidMarket, adminResolveMarket - contracts/PredictionMarketV3ManagerCLOB.sol:195-238

```

Impact: Any actor can foreclose the voiding escape hatch for an entire neg-risk event by resolving one constituent market through the permissionless `resolveEventMarket` path. If the event later requires voiding (disputed result, regulatory withdrawal, oracle misconfiguration), the admin cannot use `voidEvent` and the per-market admin remedies cannot overwrite the already-resolved market. Users who would have received partial refunds under a void instead receive all-or-nothing YES/NO payouts determined by whichever oracle outcome was locked in. The DoS

is permanent for that event: `voidEvent` has no mechanism to skip resolved markets, and the resolved state is irreversible at the manager level. No direct fund theft occurs, but the loss of the void remedy can disadvantage holders of markets that would have received non-zero void payouts.

Recommended Mitigation: Give the resolution admin a guaranteed window to void before any constituent market can be resolved, rather than making `voidEvent` tolerate already-resolved markets. Letting a void run alongside an already-resolved market is unsafe: a market resolved YES pays its holders in full, and the void then applies a fresh set of YES payouts to the remaining markets, so the combined payout exceeds the event's single-basket backing and `redeemNOPositions` can no longer recover the minted wcol.

Instead, gate both `resolveEventMarket` and `adminResolveEventMarket` behind a cooldown measured from the shared close time, so resolution cannot begin until the admin has had an opportunity to call `voidEvent`:

```
uint256 constant RESOLVE_COOLDOWN = 10 minutes;

// in resolveEventMarket and adminResolveEventMarket, before resolving the leg:
require(
    block.timestamp >= manager.getMarketClosesAt(marketId) + RESOLVE_COOLDOWN,
    "resolve cooldown"
);
```

During the cooldown the markets are closed (no trading) but not yet resolvable, so the admin can decide whether to void; once it elapses, permissionless resolution proceeds as before. Size the cooldown to give operators a realistic window to react after close. Because `voidEvent` still requires a fully-unresolved event, the void payouts always sum within the single-basket backing and no resolved-plus-voided double-payout can arise.

Myriad: Fixed in commits [f8b1080](#), [46f2696](#), and [82818a7](#)

Cyfrin: Verified. Instead of the cooldown, the fix makes `voidEvent` tolerate already-resolved legs: it skips any leg already in the resolved state and requires that `leg's yesPayouts[i] == 0`, voiding only the still-unresolved legs. A companion change makes a YES resolution atomic - both `resolveEventMarket` and `adminResolveEventMarket` force every other leg to NO and set `evt.resolved = true` in the same transaction - so while an event is unresolved, any resolved leg is necessarily NO.

This removes the foreclosure for the case that matters: an early NO resolution (an eliminated outcome) no longer reverts `voidEvent`, and the admin can still void the remaining legs. The skipped NO leg keeps its 1:1 NO payout via `redeemPosition`, so no double-payout against the basket backing can occur. A YES resolution finalizes the event by design, intentionally closing void once a determinate winner exists. The approach also preserves the protocol's required progressive pre-close resolution, which the originally-recommended cooldown would have blocked.

7.2.3 `NegRiskAdapter::resolveEventMarket` first-mover race permanently wedges a second YES-returning oracle market, blocking `resolveEvent` and redemption

Description: `resolveEventMarket` resolves a single constituent neg-risk market by calling `manager.resolveMarket(marketId)`, which permanently records the market as resolved with its oracle-reported outcome. Only after that state write does the adapter enforce the event-level "at most one YES" invariant via `_requireNoOtherYes`. Because each per-market resolution is a separate transaction, the two operations are not atomic across markets. If two constituent oracles independently return YES - an upstream invariant violation the adapter is designed to defend against, not prevent - the first market resolved passes `_requireNoOtherYes` (no other YES exists yet) and is permanently recorded as YES. Every subsequent attempt to resolve the second YES-returning market reverts inside `_requireNoOtherYes` because it finds the first market already YES; the entire transaction rolls back atomically, leaving the second market in its closed (unresolved) state. `resolveEvent` then permanently reverts at the all-markets-resolved gate (`require(getMarketState(mid) == resolved, "market !resolved")` at `contracts/NegRiskAdapter.sol:389`) because the wedged market never transitions to resolved. The downstream `redeemNOPositions`, which is gated on `evt.resolved`, is also blocked.

Files:

`NegRiskAdapter::resolveEventMarket`, `resolveEvent`, `_requireNoOtherYes` - `contracts/NegRiskAdapter.sol:329-340`, `363-402`

Impact: The permissionless resolution path for the event is bricked once two constituent oracles return YES. Holders of the wedged market cannot resolve it through any permissionless call, and the adapter's NO-position cleanup (`redeemNOPositions`) cannot run until the event is resolved. A `RESOLUTION_ADMIN_ROLE` can reconcile the state by calling `adminResolveEventMarket` with outcome NO for the wedged market (which bypasses `_requireNoOtherYes` because the forced outcome is not YES), after which permissionless `resolveEvent` succeeds. Alternatively, `adminResolveEvent` can resolve the event directly. The "at most one YES" solvency invariant is never violated - the conflicting state is rejected, not accepted - and no funds are lost. This is a recoverable, admin-mediated denial of the permissionless path; the trigger is an upstream oracle fault outside the adapter's control.

Recommended Mitigation: Encode the event's mutual-exclusivity invariant directly instead of enforcing it reactively. Because at most one outcome in a neg-risk event can win, a YES on any constituent market is by definition a NO on all the others. Make `resolveEventMarket` act on that: when `manager.resolveMarket(marketId)` returns YES, atomically force every other constituent market to NO (via `manager.adminResolveMarket(otherId, NO)`, which the adapter is authorized to call) and finalize the event in the same transaction, rather than committing one YES and then reactively rejecting a later second YES in `_requireNoOtherYes`. This makes a dual-YES state structurally impossible: once any market resolves YES the rest are already NO, so no market can be left permanently unresolvable and no admin intervention is required. The all-NO "Other won" case (`winningIndex = -1`) is unaffected, since markets still resolve NO through their own oracles and the event finalizes once all are NO.

Myriad: Fixed in commits [f8b1080](#), [46f2696](#), and [82818a7](#)

Cyfrin: Verified.

7.2.4 PR-151 deploy and interaction scripts load the broadcasting private key from a plaintext `PRIVATE_KEY env var`

Description: All four Foundry scripts introduced or modified by PR-151 read the transaction-signing key via `vm.envUint("PRIVATE_KEY")` and broadcast with `vm.startBroadcast(privateKey)`. The shell helper `scripts/create_clob_markets.sh` additionally prompts for `PRIVATE_KEY` and exports it into the process environment via `prompt_secret` before invoking `forge` script. The keys involved are privileged: `CreateNegRiskEvent` and `CreateCLOBMarket` require `MARKET_ADMIN_ROLE`; `AdminResolveNegRiskEvent` requires `RESOLUTION_ADMIN_ROLE`. A key stored in a `.env` file or exported into the process environment is readable by anything with access to the operator's machine state - CI log output that echoes environment variables, shell history, a compromised development machine, or an accidental commit of a non-gitignored copy of the file.

Files:

`script/AdminResolveNegRiskEvent.s.sol`, `script/CreateNegRiskEvent.s.sol`, `script/ResolveNegRiskEvent.s.sol`, `script/CreateCLOBMarket.s.sol`, `scripts/create_clob_markets.sh`

Impact: Exploitation requires a secondary off-chain compromise of the operator's environment; no on-chain attacker can extract the key directly. If the key does leak, the attacker gains the role it controls: `MARKET_ADMIN_ROLE` allows creating new markets; `RESOLUTION_ADMIN_ROLE` allows overriding market and event resolution outcomes. No funds are directly extractable through these roles alone, but admin resolution control can determine which outcome pays out on in-flight prediction markets.

Recommended Mitigation: Migrate all four scripts to Foundry's encrypted keystore. Replace the `vm.envUint("PRIVATE_KEY")` / `vm.startBroadcast(privateKey)` pair with a bare `vm.startBroadcast()` and invoke via `forge script ... --account <name>`, where the named account was imported with `cast wallet import`. In `scripts/create_clob_markets.sh`, replace the `PRIVATE_KEY` prompt and export with an `--account` flag passthrough so the private key is never placed in the process environment or read from a plaintext file.

Myriad: Fixed in commit [dcfe3b8](#)

Cyfrin: Verified.

7.2.5 Per-market admin mutators `adminSetClosesAt` and `updateMarketOracle` lack the neg-risk carve-out enforced on the resolution and void paths

Description: PR-151's neg-risk resolution and void paths protect event-level consistency by gating neg-risk markets to the adapter: `resolveMarket`, `adminResolveMarket`, and `adminVoidMarket` each carry `require(!market.negRisk || msg.sender == negRiskAdapter)` (contracts/PredictionMarketV3ManagerCLOB.sol:179, 201, 226-227), so per-market state changes on a neg-risk event can only happen through `NegRiskAdapter`, which keeps all constituent markets coordinated. Two other per-market admin mutators omit this carve-out and operate on a single leg with no event-level coordination:

- `adminSetClosesAt` (contracts/PredictionMarketV3ManagerCLOB.sol:261) rewrites one market's `closesAt`, even though `createEvent` requires every constituent market to share one `closesAt` (contracts/NegRiskAdapter.sol:140-142, "closesAt mismatch").
- `updateMarketOracle` (contracts/PredictionMarketV3ManagerCLOB.sol:241) re-points one market's resolution oracle; its only state check is `require(market.state != MarketState.resolved, "resolved")`. The oracle is the source of a leg's resolved outcome, so changing it changes how that leg will resolve - independently of the rest of the basket.

Files:

`PredictionMarketV3ManagerCLOB::adminSetClosesAt`, `updateMarketOracle` - contracts/PredictionMarketV3ManagerCLOB. 259, 261-273; `NegRiskAdapter::createEvent`, `resolveEventMarket` - contracts/NegRiskAdapter.sol:140-142, 329-340

Impact: Two distinct single-leg desyncs, each from a change that touches one constituent market while leaving the event uncoordinated:

1. Close desync (`adminSetClosesAt`): changing one leg's `closesAt` breaks the shared-close invariant. If a leg is closed early, or its close passes, while siblings remain open, `matchCrossMarketOrders` reverts on the closed leg, `convertPositions` and `mintAllYesTokens` revert because their `splitPosition` calls require the market open, and any user can then permissionlessly `resolveEventMarket` the closed leg - which permanently forecloses `voidEvent` for the whole event. Extending one leg instead blocks full event resolution until the latest leg closes.
2. Oracle desync (`updateMarketOracle`): a neg-risk event assumes its legs resolve consistently - at most one YES, one coherent winner across the basket. Because a leg's oracle can be replaced after creation with no event-level coordination, and two oracles can legitimately return different results for the same underlying question, changing one leg's oracle - to migrate a feed, fix a misconfiguration, or upgrade it - can cause that leg to resolve to a different outcome than the event was set up around, with no check that the new result stays consistent with the sibling markets. If the replacement oracle reports YES on a leg after another leg has already resolved YES, the event wedges permanently because the per-market resolution path rejects a second YES once one leg is YES, leaving the conflicting leg unresolvable; if it flips a leg's result, the event `winningIndex` and the downstream redemption, convert, and cross-market economics change. Either way the divergence stems from an uncoordinated per-leg oracle change rather than a coordinated event resolution.

Both are reachable through normal `MARKET_ADMIN_ROLE` lifecycle operations - adjusting a close time, migrating or correcting an oracle - so the risk is an uncoordinated single-leg change desyncing the event, not a theft vector. Some states are recoverable by re-aligning `closesAt` or restoring the oracle before the leg resolves, but a leg that has been closed early and then resolved, or resolved through a changed oracle, is irreversible. No funds are directly stolen; the harm is loss of the void remedy, disabled cross-market settlement, and a possibly-wedged or inconsistently-resolved event.

Recommended Mitigation: Apply the same neg-risk carve-out the resolution and void paths use to both mutators, so per-market changes on a neg-risk event cannot be made in isolation:

```
// in both adminSetClosesAt and updateMarketOracle, mirroring adminVoidMarket
require(!market.negRisk || msg.sender == negRiskAdapter, "use adapter for neg risk");
```

For close-time changes, pair this with a role-gated `NegRiskAdapter` entrypoint that rewrites every constituent market's `closesAt` atomically, preserving the shared-close invariant. For oracle changes on a neg-risk leg, route

them through the adapter as well, or disallow them after creation, so a leg's resolution source cannot be changed independently of the event.

Myriad: Fixed in commit [3cda4e3](#)

Cyfrin: Verified.

7.3 Informational

7.3.1 `redeemNOPositions` lost its NatSpec in the PR and the new resolution endpoints document non-obvious caller and invariant semantics inconsistently

Description: PR-151 deleted the explanatory NatSpec block above `redeemNOPositions` (the comment describing that it redeems held NO positions, burns minted wcol, and sweeps excess to treasury), leaving an admin-only, irreversible cleanup function (`noPositionsRedeemed[eventId] = true` is permanent) entirely undocumented. The function has non-obvious preconditions a caller must understand: it requires the event resolved, can only run once, and now hard-reverts on `wcolRecovered < minted` ("insolvent event payouts") - a state an operator must be able to anticipate.

```
contracts/NegRiskAdapter.sol
474: function redeemNOPositions(bytes32 eventId) external nonReentrant {
475:     require(registry.hasRole(registry.DEFAULT_ADMIN_ROLE(), msg.sender), "not admin");
476:     Event storage evt = _events[eventId];
477:     require(evt.resolved, "not resolved");
478:     require(!noPositionsRedeemed[eventId], "already redeemed");
```

Recommended Mitigation: Restore a NatSpec block on `redeemNOPositions` documenting that it is admin-only, single-shot (sets `noPositionsRedeemed`), requires the event resolved, burns exactly `mintedWcolPerEvent[eventId]`, sweeps any excess to treasury, and reverts if recovered wcol is below the minted amount (insolvency).

Myriad: Fixed in commit [783fec2](#)

Cyfrin: Verified.

7.3.2 Bad debt socialize comment is misleading

Description: In `NegRiskAdapter::redeemNOPositions`, the adapter records `wcol` gained during the redeem loop as `wcolRecovered` and compares it to `mintedWcolPerEvent[eventId]`. The code reverts when recovery is short:

```
// Never silently socialize bad debt.
require(wcolRecovered >= minted, "insolvent event payouts");
```

The comment suggests holders of wrapped collateral would absorb a shortfall across the entire wcol supply. That is not how `WrappedCollateral` behaves. `adapterMint` increases wcol total supply without depositing underlying, but each `unwrap` still burns one wcol and transfers one underlying from the shared vault. Holders who redeem before the pool is stressed generally receive the full 1:1 amount.

The practical failure mode is vault insolvency: where the wrapper would hold less underlying than outstanding supply. The users most exposed are late `unwrap` callers who may hit `safeTransfer` failures when underlying runs out, not a broad socialization of bad debt across all wcol balances.

The check therefore mitigates insolvency of the wcol vault

Recommended Mitigation: Replace the comment with language that the `require` prevents insolvency of the wcol vault.

Myriad: Fixed in commit [aaaf2e1](#)

Cyfrin: Verified.

7.3.3 `NegRiskAdapter::convertPositions` routes a converter's deposited NO stake to the treasury on an all-NO event resolution

Description: `convertPositions` lets a holder swap amount of NO(`noOutcomeIndex`) for amount of YES in each of the other `n-1` markets, minting `(n-1) * amount` unbacked wcol tracked in `mintedWcolPerEvent`. The converter's transferred-in NO(`noOutcomeIndex`) is retained by the adapter as backing. If the event later resolves to "Other" - all markets NO, `winningIndex = -1` - every YES the converter received is worthless, while the converter's original NO(`noOutcomeIndex`) is itself a winning position on an all-NO resolution. That NO is redeemed

by the adapter inside `redeemNOPositions`; because it was transferred in rather than `adapterMinted`, it is not part of minted, so its recovered value surfaces as `excess = wcolRecovered - minted` and is swept to the treasury (contracts/NegRiskAdapter.sol:513-516).

The net effect: a converter who would have been paid out had they simply held `NO(noOutcomeIndex)` instead forfeits that stake to the treasury by converting, specifically on the all-NO branch. On a named-winner resolution the converter breaks even (`excess` is zero) or the loss accrues to the winning outcome's holders; only the all-NO outcome routes the converter's stake to the protocol. Value is conserved and no third party can extract it, but the economic asymmetry is non-obvious and undocumented.

Recommended Mitigation: Document that `convertPositions` is a one-way position swap whose supplied NO stake is forfeited to the treasury if the event resolves all-NO, so integrators and users can price the trade-off. If returning that residual to the converter rather than the treasury is intended instead, `redeemNOPositions` would require per-converter accounting of transferred-in NO - a substantial design change; at minimum, surface the asymmetry in the `convertPositions` NatSpec and user-facing docs.

Myriad: Fixed in commit [5173aa6](#)

Cyfrin: Verified.

7.3.4 `winningIndex -2` wrong after void

Description: At `NegRiskAdapter::createEvent`, the adapter sets `winningIndex = -2` with comment `unresolved sentinel` while `resolved` is false. That pairing is coherent.

`NegRiskAdapter::voidEvent` also sets `winningIndex = -2`, but it sets `resolved` to true and emits `EventVoided` rather than `EventResolved`. After void, the event is finalized with custom per-market payout ratios on every leg. The stored index still reads as the same unresolved sentinel used at creation.

Integrators that call `NegRiskAdapter::getEvent` and branch on `winningIndex` alone can misclassify a voided event. Logic that treats `-2` as “still open” or “not finalized” will run against a resolved event.

This can impact third-party indexers, frontends, and composable contracts that encode resolution state from `winningIndex` alone can show the wrong event status, skip void-specific redemption flows, or apply winner-based logic to a voided event.

Recommended Mitigation: Document `-2` for both pre-resolution and void in struct NatSpec. Also document to integrators to read `resolved` and `EventVoided`, not `winningIndex` alone.

Myriad: Fixed in commit [355db63](#)

Cyfrin: Verified.

7.4 Gas Optimization

7.4.1 NegRiskAdapter::voidEvent redundantly re-writes winningIndex with the unchanged -2 sentinel

Description: voidEvent writes evt.winningIndex = -2 (contracts/NegRiskAdapter.sol:465) immediately after setting evt.resolved = true. That write never changes the slot's value. createEvent already initializes evt.winningIndex = -2 (contracts/NegRiskAdapter.sol:152), and the only functions that change winningIndex are resolveEvent and adminResolveEvent - both of which set evt.resolved = true. Because voidEvent is guarded by require(!evt.resolved, "already resolved"), any event reaching it is still unresolved and therefore still holds the create-time -2. The assignment is a no-change SSTORE (storing -2 over -2), costing a cold storage access with no effect.

```
contracts/NegRiskAdapter.sol
152:     evt.winningIndex = -2; // unresolved sentinel    (set at creation)
464:     evt.resolved = true;
465:     evt.winningIndex = -2;                          // re-writes the unchanged value
```

Recommended Mitigation: Delete the redundant assignment in voidEvent; the -2 sentinel is already in place from createEvent for any unresolved event. To keep the void semantics self-documenting without the wasted write, leave a short comment in its place:

```
evt.resolved = true;
// winningIndex retains its createEvent sentinel (-2); voided events are
// distinguished from unresolved ones by evt.resolved, not winningIndex.
```

This removes one no-change SSTORE from each voidEvent call.

Myriad: Fixed in commit [58a4a8f](#)

Cyfrin: Verified.